GSOC Proposal: MIT APP INVENTOR, 2025

Assets Library: Enable Seamless Asset Management in App Inventor

Lakshya Shishir Khandelwal

Indian Institute of Technology, Roorkee

Uttarakhand, India

Table of Contents

About Me

Personal Info

- Name: Lakshya Shishir Khandelwal
- Email: lakshyashishir1@gmail.com
- GitHub: https://github.com/lakshyashishir
- Mentors: Evan Patton, Jose Dominguez, Jeff Schiller
- Location: India
- Time zone: IST (UTC +05:30)

Educational Info

- University: Indian Institute of Technology, Roorkee
- Major: Geophysical Technology
- Current Year: Junior (3rd) (Expected Graduation: 2027)
- Degree: Integrated Master of Technology (Integrated MTech)

Background

My passion for coding ignited in 8th grade with MIT App Inventor, where I built apps like "Study Time Management," earning finalist and winner titles in Google Code to Learn. This early experience sparked my journey into AI and blockchain, leading me to win multiple hackathons, including Second Track in Move on Aptos Hackathon 2024 and pool prizes at Eth Singapore and Eth India. Currently, as a developer at SDSLabs, IIT Roorkee's premier technical group, I work on production-grade applications using React, JavaScript, and Python, focusing on next-generation AI products and scalable systems. My interests in front-end development, system design, and AI-driven solutions drive my enthusiasm for the Assets Library project.

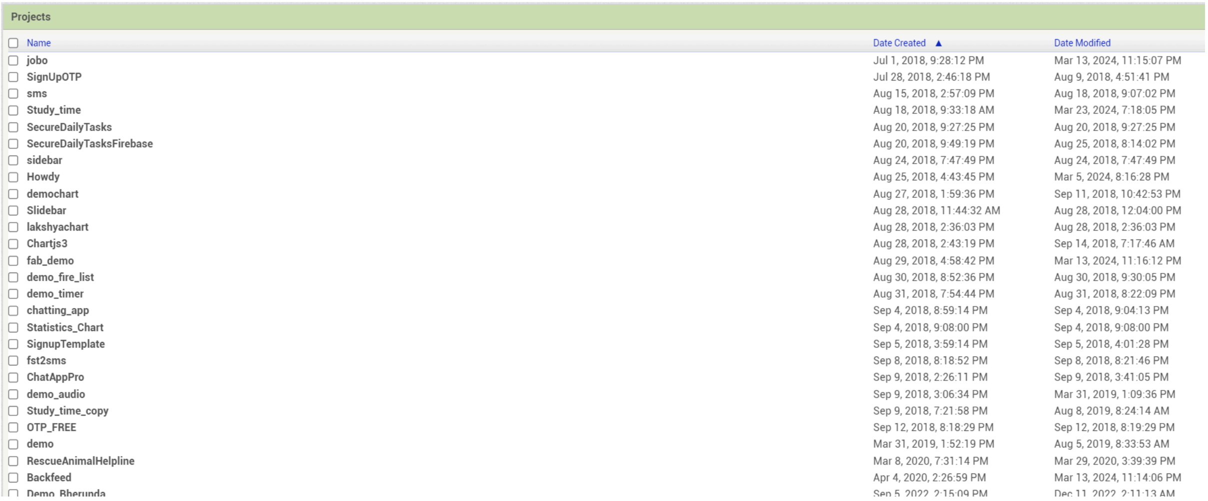
Programming Languages

React.js, JavaScript, Node.js, Python, Go, TypeScript

Why MIT App Inventor?

My deep bond with MIT App Inventor began at the start of my programming journey, igniting my curiosity and enabling me to bring ideas to life through app development. Throughout high school, it became pivotal in my projects, fostering creativity and fueling my passion for innovation.

As I join Google Summer of Code, I'm eager to give back to the community by reconnecting with MIT App Inventor and contributing to its success. I aim to be an active member, empowering others to explore app development.



Projects

Name	Date Created ▲	Date Modified
jobo	Jul 1, 2018, 9:28:12 PM	Mar 13, 2024, 11:15:07 PM
SignUpOTP	Jul 28, 2018, 2:46:18 PM	Aug 9, 2018, 4:51:41 PM
sms	Aug 15, 2018, 2:57:09 PM	Aug 18, 2018, 9:07:02 PM
Study_time	Aug 18, 2018, 9:33:18 AM	Mar 23, 2024, 7:18:05 PM
SecureDailyTasks	Aug 20, 2018, 9:27:25 PM	Aug 20, 2018, 9:27:25 PM
SecureDailyTasksFirebase	Aug 20, 2018, 9:49:19 PM	Aug 25, 2018, 8:14:02 PM
sidebar	Aug 24, 2018, 7:47:49 PM	Aug 24, 2018, 7:47:49 PM
Howdy	Aug 25, 2018, 4:43:45 PM	Mar 5, 2024, 8:16:28 PM
demochart	Aug 27, 2018, 1:59:36 PM	Sep 11, 2018, 10:42:53 PM
Slidebar	Aug 28, 2018, 11:44:32 AM	Aug 28, 2018, 12:04:00 PM
lakshyachart	Aug 28, 2018, 2:36:03 PM	Aug 28, 2018, 2:36:03 PM
Chartjs3	Aug 28, 2018, 2:43:19 PM	Sep 14, 2018, 7:17:46 AM
fab_demo	Aug 29, 2018, 4:58:42 PM	Mar 13, 2024, 11:16:12 PM
demo_fire_list	Aug 30, 2018, 8:52:36 PM	Aug 30, 2018, 9:30:05 PM
demo_timer	Aug 31, 2018, 7:54:44 PM	Aug 31, 2018, 8:22:09 PM
chatting_app	Sep 4, 2018, 8:59:14 PM	Sep 4, 2018, 9:04:13 PM
Statistics_Chart	Sep 4, 2018, 9:08:00 PM	Sep 4, 2018, 9:08:00 PM
SignupTemplate	Sep 5, 2018, 3:59:14 PM	Sep 5, 2018, 4:01:28 PM
fst2sms	Sep 8, 2018, 8:18:52 PM	Sep 8, 2018, 8:21:46 PM
ChatAppPro	Sep 9, 2018, 2:26:11 PM	Sep 9, 2018, 3:41:05 PM
demo_audio	Sep 9, 2018, 3:06:34 PM	Mar 31, 2019, 1:09:36 PM
Study_time_copy	Sep 9, 2018, 7:21:58 PM	Aug 8, 2019, 8:24:14 AM
OTP_FREE	Sep 12, 2018, 8:18:29 PM	Sep 12, 2018, 8:19:29 PM
demo	Mar 31, 2019, 1:52:19 PM	Aug 5, 2019, 8:33:53 AM
RescueAnimalHelpline	Mar 8, 2020, 7:31:14 PM	Mar 29, 2020, 3:39:39 PM
Backfeed	Apr 4, 2020, 2:26:59 PM	Mar 13, 2024, 11:14:06 PM
Demo_Bherunda	Sep 5, 2022, 2:15:09 PM	Dec 11, 2022, 2:11:13 AM

(Some projects I built on MIT App Inventor in my school time)

Why me?

With extensive experience in open-source contributions and a strong background in front-end development, I bring expertise in React, JavaScript, Node.js, and AI/ML, honed over years of coding. My work at SDSLabs, building production-level applications, and my success in blockchain hackathons (e.g., Move on Aptos 2024, Eth Singapore/India, EigenLayer Infinite Hackathon) equip me to tackle technical challenges and deliver a robust Assets Library, integrating seamlessly with App Inventor’s ecosystem.

Achievements

Google Code to Learn Winner, Google India: 2020, 2021

Google Code to Learn Finalist, Google India: 2019

Syntax Error 2023: Finalist

Winter of Code 2023: Accepted

Move on Aptos Hackathon 2024: Second Track Winner

Eth Singapore and Eth India Hackathons: Pool Prize Winner

EigenLayer Infinite Hackathon: Gwei Standing Winner (Team)

Past Projects /Experience

- Evolve Network- 05/2024 - 01/2025 -Built a decentralized AI platform to crowdsource computing and deploy AI agents, RAG applications, and custom datasets, incentivized by the Evolve token. Launched in January 2025, now serving thousands of monthly active users.
- xPay - 03/2024 - 04/2024 - Interned as a Frontend Engineer, contributing to a world-class payment orchestration product. **Tech Stack: Javascript, Java**
- Quizio - 08/2023 - 01/2025 - I contributed to developing the front end for quiz-taking and checking processes, as well as debugging the backend. This project provided hands-on experience with a large-scale CRUD application built using **JavaScript, ReactJS, and MongoDB**

Frontend : https://github.com/sdslabs/Helios
Backend : https://github.com/sdslabs/Athena

- LeSo - 08/2023 - 03/2024 - Single-handedly designed and developed an in-production web app for a Legal Tech startup, pivoting it into a visa booking platform. Founders now run a sustainable business with the app I built. Tech Stack: JavaScript, ReactJS, MongoDB. **Tech Stack: JavaScript, ReactJS, MongoDB**

- Study Time Management- School Project - One of my **App Inventor** projects I built in school. Became a finalist in Google Code to Learn with the same project. Continued to build complex apps on the platform even after this.
  AIA :
  https://drive.google.com/file/d/1lkrgvlF6FXrdREIAY1rXY2CGH9s_fkYf/view?usp=sharing

Project Details

Overview

This GSoC proposal aims to create an Assets Library for MIT App Inventor, enabling users to upload, organize, and import assets (images and sounds) across projects via a centralized interface in the Designer. Building on my discussion with mentors, I'll implement a responsive panel distinct from the AssetListBox, leverage the existing UserFileData structure for user-specific storage, and ensure robust validation by refactoring existing upload logic. This feature will enhance workflow efficiency for educators, students, and developers, aligning with App Inventor's mission to simplify app creation.

Project Scope (175 Hrs)

This medium-sized 175-hour project focuses on:

Assets Library Development: Design a new UI panel and backend system for uploading, organizing, and importing assets, with user-specific storage and validation.

User Stories

To illustrate the Assets Library’s value, here are key user scenarios:

Educator Scenario:
"As a computer science teacher creating multiple app-building tutorials, I want to store my commonly used educational images and sound effects in a central library so I can quickly access them across different project tutorials without re-uploading each time."

Student Scenario:
"As a student working on several related app projects, I want to manage my game sprites and sound effects in one place so I can easily import the same assets into multiple projects and maintain consistency across my applications."

Developer Scenario:

"As an app developer creating branded applications, I want to organize my UI elements, logos, and brand sounds in categorized folders so I can efficiently reuse these assets across client projects while keeping them well-organized."

These scenarios highlight the library's utility for reuse, organization, and efficiency, tailored to App Inventor's diverse user base.

UX Flow

The user experience for the Assets Library is designed with intuitive interaction patterns, emphasizing Evan's focus on user-facing design:

1. Access and Navigation:
    - Users access the Assets Library via a new tab or panel in the Designer interface.
    - The panel features a responsive layout with scrollbars for large asset sets, adapting to various screen sizes.

   Navigation includes:
    - Main library view (toggleable grid or list layout).
    - Sidebar for folder navigation.
    - Top bar with search and filter controls, plus a prominent "Upload" button.

2. Asset Upload Flow:

   User clicks "Upload" → Selects file(s) → System validates (type/size) → Success/Error feedback → Assets appear in the library grid.

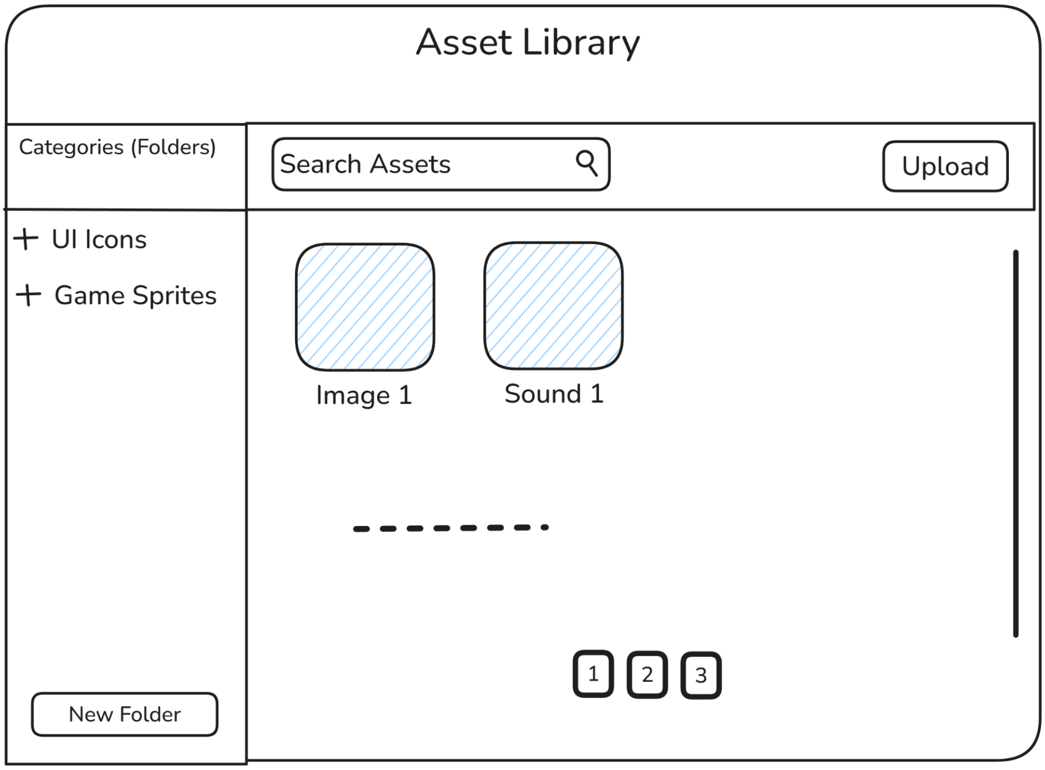
3. Asset Organization Flow:
   User selects assets → Creates folder/tag → Names it → Moves assets → Visual confirmation in the sidebar and grid.

4. Asset Import Flow:
   User selects asset(s) → Clicks "Add to Project" → Sees confirmation dialog → Assets imported → Visual update in project's AssetListBox.

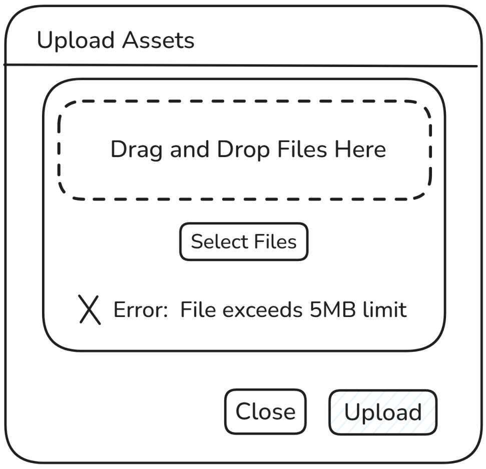
Wireframes

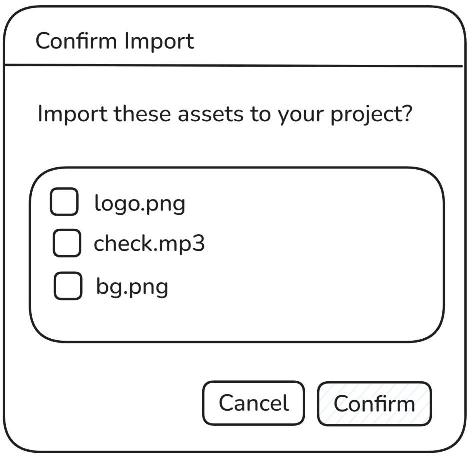
These initial wireframes illustrate the Assets Library UI, kept simple for discussion and refinement with mentors, focusing on three key views.

1. Overview of the Assets Library with folder navigation, search, and asset display.



2. Modal dialog for uploading assets with drag-and-drop and error feedback.





3. Confirmation dialog for importing selected assets into a project. I am thinking of a similar diagram like library to select files, before this dialog.

Implementation Details and Deliverables

In this section, I will be going over the main implementation steps of this project.

1. Asset Upload Interface

➢ UI Implementation:
  - Develop a responsive panel using GWT and UiBinder, extending DecoratorPanel and TabPanel from the Designer codebase.
  - Add drag-and-drop support via HTML5 File API with GWT's JSNI.
  - Include scrollbars for large asset sets using ScrollPanel widgets.

➢ Validation:
  - Refactor validation logic from UploadServlet.java into a reusable ValidationService class.
  - Check MIME types (PNG, JPG for images; MP3, WAV for sounds) and enforce a 5MB size limit, displaying errors via ErrorReporter.

➢ Backend Processing:
  - Extend UploadServlet to process library uploads with a new parameter distinguishing them from project uploads.
  - Use existing chunking mechanisms for large files, storing assets in UserFileData.

2. Storage Architecture

Following analysis of the App Inventor codebase (MIT App Inventor Public Open Source), the storage solution will:

➢ UserFileData Approach: Extend the existing UserFileData structure (used for signing keys and backpack data) to manage user-level assets, as suggested by Evan Patton.

➢ GCS Storage: Store assets in Google Cloud Storage under users/{user_id}/assets/{folder}/{asset_filename}, where {user_id} is the user's account identifier, {folder} is a user-defined folder or tag, and {asset_filename} is sanitized for uniqueness. This reuses the existing GCS bucket with a flexible structure.

- Metadata: Store asset metadata (name, type, folder, upload date) in the App Engine datastore, creating indexes for efficient retrieval.
- Integration: Extend UploadServlet and DownloadServlet endpoints to handle library assets, maintaining architectural consistency.

3. Asset Organization

To manage assets effectively:

- Categorization UI:
  - Implement folders/tags with GWT's Tree widget in the sidebar.
  - Create a grid view with image thumbnails using CellList and custom renderers, plus sound previews via HTML5 audio elements with JSNI.
- Storage Implementation:
  - Extend UserFileData to associate assets with user accounts, storing folder metadata in the App Engine datastore.
- Operations:
  - Enable drag-and-drop between categories, with context menus for rename/delete/move and batch operations.

4. Asset Import Functionality

- Import UI:
  - Add an "Add to Project" button to asset cards, with multi-select for batch imports and a confirmation dialog.
- Backend Transfer:
  - Extend FileExporter to copy assets from user storage to project storage as logical references (not physical copies unless modified), updating project metadata.
  - Optimize transfers with existing chunking mechanisms.
- Integration:
  - Ensure imported assets refresh in AssetListBox via the event bus system, with success/failure notifications.

5. User Experience Enhancements

- ➢ Search and Filter:
  - ○ Implement a search bar with GWT’s TextBox and a custom SearchService querying asset names and categories.
  - ○ Add filters using CheckBox widgets for asset type (image/sound) and folder, collapsing responsively on small screens.
- ➢ Error Handling:
  - ○ Use ErrorReporter for consistent error reporting, with retry logic for failed uploads and fallbacks for unsupported browsers.
- ➢ Documentation:
  - ○ Add an in-app guide with HTML tooltips and contextual help for first-time users, including keyboard shortcuts.

Feature	Description
Asset Upload Interface	Responsive panel for drag-and-drop uploads, with type/size validation
Asset Organization	Folders/tags, grid view with previews, tied to UserFileData
Asset Import	Button and batch import into projects
Validation	Refactored validation from existing upload logic
User Experience	Search, filters, error messages

Technical Challenges

Some challenges I suspect we can encounter, during the implementation:

1. Cross-Project Management: Use reference-counting in UserFileData, logical references for imports.
2. Performance: Pagination (30-50 assets), thumbnail caching, lazy-loading with LazyPanel.
3. Consistency: Synchronization service with locking, periodic checks.
4. Compatibility: HTML5 API detection, file input fallbacks, mobile testing.
5. Access Control: Validate {user_id} in UploadServlet/DownloadServlet with UserInfoService.

Project Timeline

Following is the project timeline for my GSoC. I would be taking up the 175 hour format.

Pre GSoC Period	
April 7th - May 7th	● Study Designer codebase (UploadServlet.java, UserFileData.java) and asset storage. Contribute to small open issues, building familiarity with the community and mentors.

Community Bonding Period	
May 8th - June 1st	● Engage with the App Inventor community on forums and discuss implementation with mentors. ● Finalize UserFileData approach, develop wireframes/integration diagrams. ● Explore relevant code, focusing on Designer and media modules.

Coding Period	
June 2nd - June 15th	● Milestone: Backend Foundation ● Implement UserFileData extension for asset storage, modify UploadServlet for library uploads, create metadata model (name,

	type, folder, upload date) in datastore, and write JUnit tests.
June 16th - June 29th	• Milestone: Upload Interface • Design Assets Library panel with GWT/UiBinder, implement drag-and-drop uploads, create grid/list view, and integrate with backend.
June 30th - July 13th	• Milestone: Organization Features • Develop organization features (folders/tags with Tree widget), implement initial search (TextBox/SearchService) and filters (CheckBox widgets), and test UI responsiveness.
July 14th - July 27th	• Milestone: Import and Midterm Prep • Create import mechanism with FileExporter, add batch operations, demo upload/browsing/organization for midterm evaluation (July 14th - 18th), refine based on feedback.
July 14th - July 18th	• **Evaluation 1 Phase:** Demo upload and organization features, submit progress report.
July 28th - August 10th	• Milestone: UX Enhancements • Enhance UX with error handling (ErrorReporter), previews, in-app documentation, optimize for large libraries (pagination, lazy-loading), and address compatibility issues.

August 11th - August 24th	● Milestone: Final Testing and Wrap-Up ● Conduct end-to-end testing with Selenium WebDriver, document implementation (architecture, user guides), and prepare final report/presentation for submission by August 25th.
August 25th - September 1st	● Final Week: Submit final work product and mentor evaluation by September 1st, incorporating any last-minute feedback.

Post GSoc Goals

Post-GSoC, I'll continue contributing to MIT App Inventor, refining the Assets Library based on user feedback and exploring related enhancements. I aim to mentor future contributors, sharing my experience to strengthen the community.

Development Environment

Windows with manual App Inventor setup.

Tools: VS Code for JavaScript/Java, Git for version control, local App Engine server, Chrome DevTools for UI debugging.

Time Commitment

May 6th - May 15th: College end-semester exams (10-15 hours/week)

May 15th - July 15th: Summer vacation (30 hours/week)

July 16th - August 27th: College resumes (25 hours/week)

I am 100% dedicated to MIT App Inventor and have absolutely no plans to submit proposals to any other organizations.